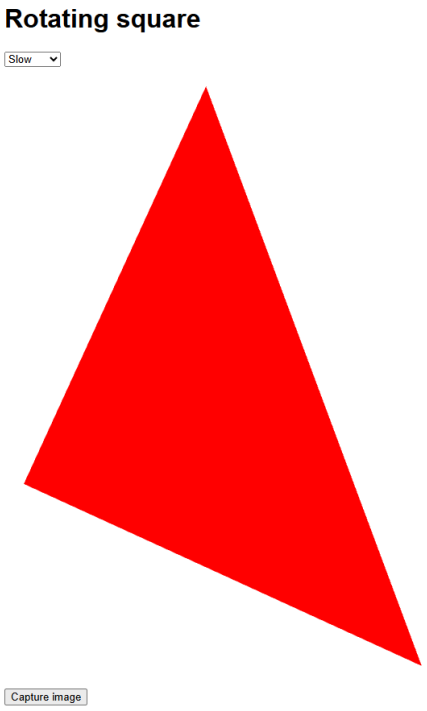
A1



changing num_strip_vertices in gl.drawArrays(gl.TRIANGLE_STRIP, 0, num_strip_vertices) to 3 (A1.1) makes gl.drawArrays only read the first 3 coordinates from the vertices array(A1.2), making the shape change from a square into a triangle

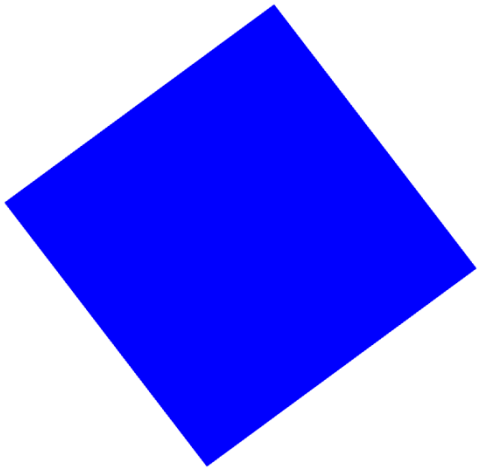
A1.1:

```
// draw triangle strip
let num_strip_vertices = vertices.length;
gl.drawArrays(gl.TRIANGLE_STRIP, 0, 3);
```

A1.2:

```
vertices = [[0,1], [-1,0], [1,0], [0,-1]];
```

A2



by changing the third value in gl_FragColor to 1.0 and the first value to 0.0 the square's colour changed to blue. This is because the 4 values are red, green, blue, and opaque so by changing red to 0 and blue to 1 the square becomes blue.

```
precision mediump float;

vec3 red = vec3(1.0, 0.0, 0.0);
vec3 green = vec3(0.0, 1.0, 0.0);
vec4 yellow = vec4(red + green, 1.0);

// A4: ADD CODE HERE

void main()
{
    // A2 & A4: MODIFY BELOW

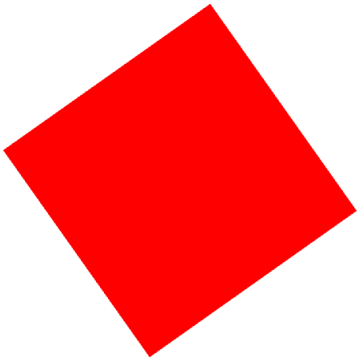
    gl_FragColor = yellow;
}
```

I defined red green and yellow, red and green as vec3 as they don't need them to contain an opaque value and then I defined yellow as a vec4 adding red and green inside of it to create the colour yellow then adding the opaque value on the end, then finally changed gl_FragColour to be set to yellow.

A3

```
gl_Position.x *= (1.0 + s) / 2.0;
gl_Position.y *= (1.0 + s) / 2.0;
```

As the shape rotates and theta is increased by theta_step (by default it is set to 0.01 and is increased/ decreased by the drop down menu), the x and y position of the shape is then timesed and set by (1.0 + s) / 2.0 where s is sin(theta) so as theta increases by 0.01 each frame it cause the x and y position to grow and shrink at the same rate causing the entire shape to grow and shrink. This also only happens while the shape is not paused in the drop down menu as theta_step gets changed to 0.

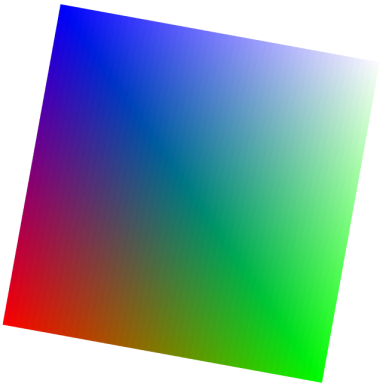


A4

gl.vertexAttribPointer(colour_loc, 4, gl.FLOAT, false, 0, 0) tells webGL to use the data in the colour_buf when the shader asks for colour,

```
colours = [
    [1.0,  0.0,  0.0,  1.0], // red
    [0.0,  1.0,  0.0,  1.0], // green
    [0.0,  0.0,  1.0,  1.0], // blue
    [1.0,  1.0,  1.0,  1.0]  // white
];
```

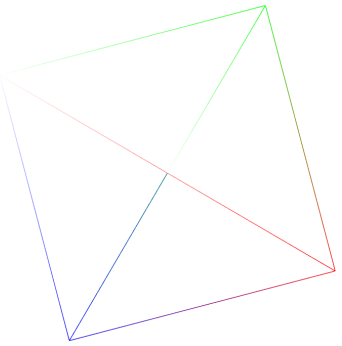
the data in the colour_buf is stored as a 2D array called colours.



gl.enableVertexAttribArray(colour_loc) enables gl.vertexAttribPointer() and uses the colour_buf that we connected to vertex_colour attribute in the shader.

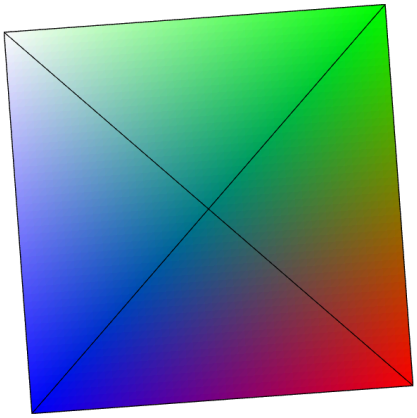
A5

The difference between drawArray and drawElement is that draw array requires use of the vertex data in the order in which you want to use it. While drawElement allows you to reuse vertex data but have to provide an index array to specify the order you want to use the vertices. In the case of drawing the wire square, indices is our index array specifying which vertices to draw lines between. E.g 0,1 will draw a line between vertex 0 and 1. In indices 0,3 and 1,2 are the diagonal lines and the other 4 pairs are the 4 outer sides of the square.



```
indices = [
    0,1,
    1,3,
    3,2,
    2,0,
    0,3,
    1,2
    ];
```

A6



For the two triangles and not the black outer lines the vertexAttributePointer's offset starts at 0 meaning when reading through the colour buffer it starts at the red. Since the two triangles only use the first 4 colours from the colour buffer as they only have 4 vertices it will not read the newly added 6 black colours that have been added. The 6 black colours added at the end will be used by the

```
colours = [
    [1.0,  0.0,  0.0,  1.0], // red
    [0.0,  1.0,  0.0,  1.0], // green
    [0.0,  0.0,  1.0,  1.0], // blue
    [1.0,  1.0,  1.0,  1.0],  // white
    [0.0,  0.0,  0.0,  1.0], // black
    [0.0,  0.0,  0.0,  1.0], // black
    [0.0,  0.0,  0.0,  1.0], // black
    [0.0,  0.0,  0.0,  1.0], // black
    [0.0,  0.0,  0.0,  1.0], // black
    [0.0,  0.0,  0.0,  1.0] // black
];
```

drawElement for each of the lines, the reason for 6 of them is that there are 6 pairs in indices meaning that all of the lines will be black and since they are all black any interpolating will just spread the same colour as there is no need to mix and make any new colours like in the two triangles. To ensure that the lines make sure to read from the black colours and are all black the offset is then changed to be let black_offset = vertices.length * 4 * 4; which is 64.

```
let black_offset = vertices.length * 4 * 4;

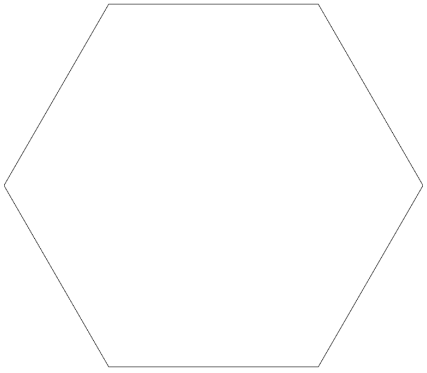
gl.vertexAttribPointer(colour_loc, 4, gl.FLOAT, false, 0, black_offset);
```

B1

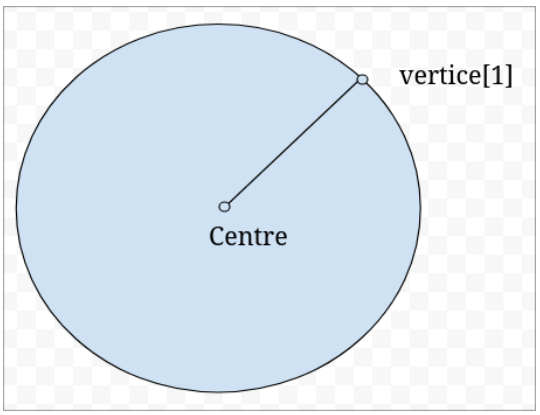
```
ertices = [];
ndices = [];
/ B1: ADD CODE HERE
onst num_vertices = 6;
or (let i = 0; i < num_vertices; i++) {
   let angle = 2 * Math.PI * i / num_vertices; // makes a cir
   vertices.push([Math.cos(angle), Math.sin(angle)]);
   indices.push(i);

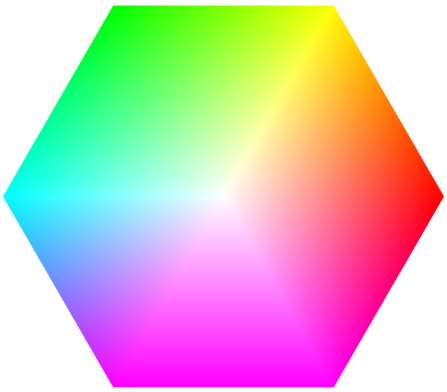
ertices.push([0.0, 0.0]); // center vertex
ndices.push(0); // center vertex index
```

Turning the pseudo code into JS code I started by defining num_vertices, then created a for loop to iterate the number of vertices which is set to 6. And the during each iteration it performs angle = 2 * Math.PI * i / num_vertices; witch calculates the angle for each of the 6 vertices from the center splitting the circle up into 6 sections, then vertices.push([Math.cos(angle), Math.sin(angle)]) calculates where the angle would intersect the circle's edge and then pushes the vertex to the array vertices. And the we push i to the array indices, when the loop is finished indices will look like [0,1,2,3,4,5,0] which is the order in which the vertices will be connected. And once the loop is finished we push 0.0,0.0 to the vertices array which marks the center and 0 to the indices array which fully connect all of the vertices completing the hexagon.



B2





When colouring the hexagon I needed to add all of the colours to the *colours* array. There were 7 colours total, one for each of the vertices including the center vertex which was white. In order to make use of the white I had to add the center vertex to my indices array, originally i didn't add this to my indices array cause it was not needed to draw the outer shape. I had a few issues adding it first because I was adding it after the indices array had been constructed, but then I realised I had to place before my for loop, once finished the colour of the shape matched the example.

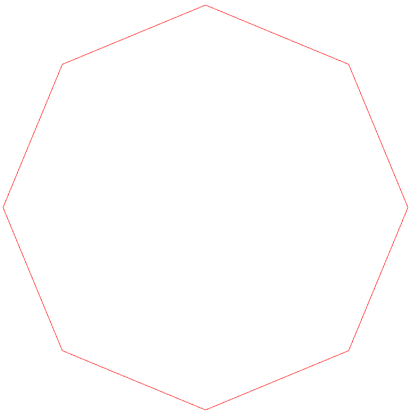
```
colours = [
    [1.0, 0.0, 0.0, 1.0], // RED
    [1.0, 1.0, 0.0, 1.0], // YELLOW
    [0.0, 1.0, 0.0, 1.0], // GREEN
    [0.0, 1.0, 1.0, 1.0], // CYAN
    [1.0, 0.0, 1.0, 1.0], // BLUE
    [1.0, 0.0, 1.0, 1.0], // MAGENTA
    [1.0, 1.0, 1.0, 1.0] //WHITE
];
```

```
const num_vertices = 6;
indices.push(num_vertices); // center vertex index

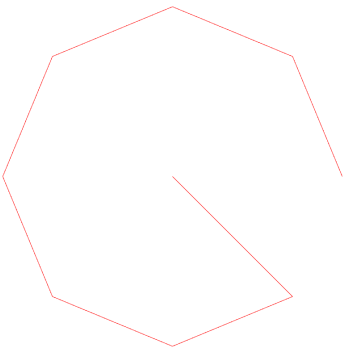
for (let i = 0; i < num_vertices; i++) {
    let angle = 2 * Math.PI * i / num_vertices; // make
    vertices.push([Math.cos(angle), Math.sin(angle)]);
    indices.push(i);
}

vertices.push([0.0, 0.0]); // center vertex
indices.push(0); // center vertex index
```

C1



To make the octagon re-using the code for part B I removed the *indices.push()* as we are using *gl.drawArrays* instead of *gl.drawElements()* so we only the vertices also needed to add *colour.push([1,0,0,1])* to the loop for each vertex to make them red and finally had to change *num_vertices* to 8. After implementing the changes it did not work. It was very close but instead of fully completing the octagon it had a line going to the centre vertex. After going through my code I realised that I had left a line of code in from when i had copied it over *vertices.push[0,0]* which made it draw the line to the center instead of completing it, once I removed the line it did not draw anything at all so I checked the console and it said that vertices only provided 8 vertices instead of 9, so I added *vertices.push[vertices[0]]* so that the



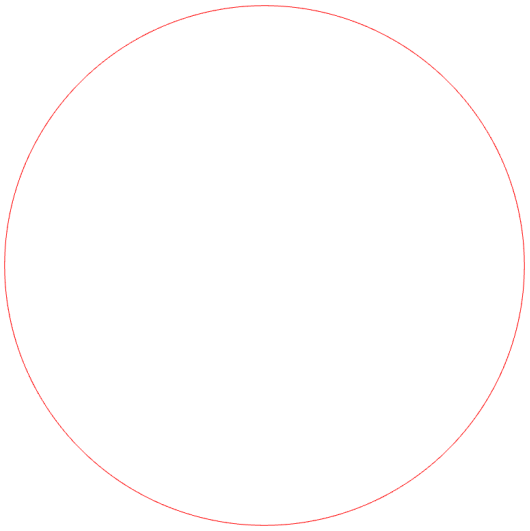
octagon would connect the last and first vertices completing the octagon. I also each of the vertices by 0.99 so that they would fit in the canvas better (like instructed).

```
const num_vertices = 8;
colours.push([1,0,0,1]) // red

for (let i = 0; i < num_vertices; i++) {
    let angle = 2 * Math.PI * i / num_vertices; // makes a circle and
    vertices.push([0.99*Math.cos(angle),0.99*Math.sin(angle)]);
    colours.push([1,0,0,1])
}

vertices.push(vertices[0]); // close the loop by duplicating the firs
colours.push([0,0,0,1]) // center vertex colour
```

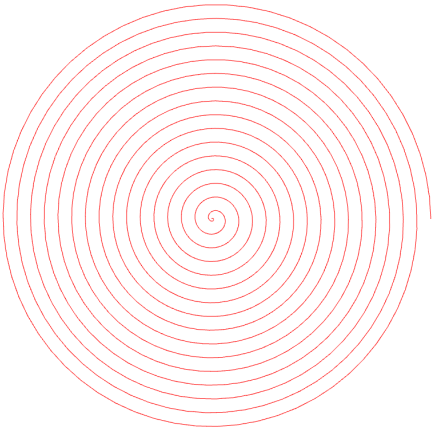
C2



Changing the num_vertices to 1000 creates so many vertices that when it renders it looks like a circle but it is in fact made up of many straight lines.

C3

To make the spiral I first created a variable called *scale* which I set to 0.99, next I changed *vertices.push([0.99*Math.cos(angle),0.99*Math.sin(angle)]);* to use the scale variable instead of 0.99 so that I could change the scale value while looping. At the end of the for loop I added *scale -= 0.001* so that the gaps between each line would be uniform, after everything had been added I had to do some clean up to ensure that the code would not try and connect the first and last vertices, so I removed *vertices.push([0,0])* as well as the +1 from num_vertices



in *gl.drawArrays()* as it no longer needed to iterate one extra time to close the shape.

```
gl.drawArrays(gl.LINE_STRIP, 0, num_vertices);
```

```
// data for attributes
vertices = [];
colours = [];
var scale = 0.99;
// C1, C3, C4, C5: ADD CODE HERE

colours.push([1,0,0,1]) // red

for (let i = 0; i < num_vertices; i++) {
   let angle = 2* 16 * Math.PI * i / num_vertices; // makes a circle and divides it
   vertices.push([scale*Math.cos(angle),scale*Math.sin(angle)]);
   colours.push([1,0,0,1])
   scale -= 0.001; // scale down for spiral effect
}

//vertices.push(vertices[0]); // close the loop by duplicating the first vertex
colours.push([0,0,0,1]) // center vertex colour
```

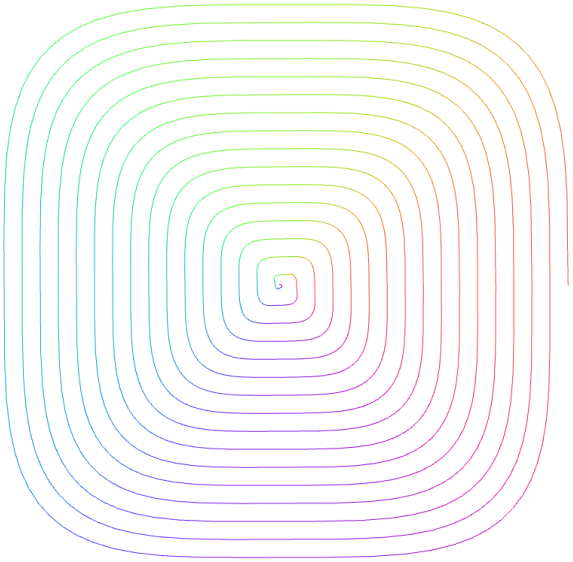
C4



First I read through *rgba_wheel()* to understand how to use it, after understanding that I need to provide it with a value *t* and then it would generate me a colour based on that it was fairly simple to implement just swapping out *colour.push[1,0,0,1]* for *colour.push[rgb_wheel(angle)]* so that each vertex would have a different colour generated instead of just giving it the colour red.

```
for (let i = 0; i < num_vertices; i++) {
   let angle = 2* 16 * Math.PI * i / num_vertices; // makes a circle and divides it
   vertices.push([scale*Math.cos(angle),scale*Math.sin(angle)]);
   colours.push(rgba_wheel(angle));
   scale -= 0.001; // scale down for spiral effect
}
```

C5



I created my squirl function using the maths provided taking t and n as inputs, t being angle that was calculated for each vertex and n being 4 as n > 2 otherwise it will just be a normal circle (link to the wiki page where it states this), and then returning x and y in the forum [x,y]. Once I had the function I replaced *vertice.push ([scale*Math.cos(angle), scale*Math.sin(angle)])* with *vertices.push([scale*squircle(angle, 4)[0], scale*squircle(angle, 4)[1]]); then using the same code as previous for the colour it was finished.*

```
function squircle(t,n){
    let x = Math.sign(Math.cos(t)) * Math.abs(Math.cos(t))**(2/n);
    let y = Math.sign(Math.sin(t)) * Math.abs(Math.sin(t))**(2/n);
    return [x,y];
}
```

```
for (let i = 0; i < num_vertices; i++) {
    let angle = 2* 16 * Math.PI * i / num_vertices; // makes a circle and divides it
    vertices.push([scale*squircle(angle, 4)[0], scale*squircle(angle, 4)[1]]);
    colours.push(rgba_wheel(angle));
    scale -= 0.001; // scale down for spiral effect
}
```